Dataset Format

1) **Alpaca Format (Instruction Tuning):**

Format

```
[
  {
    "instruction": "...",
    "input": "...",
    "output": "..."
  },
  {
    "instruction": "...",
    "input": "...",
    "output": "..."
  }
]
```

When to Use Alpaca

Used for **Supervised Fine-Tuning (SFT)** to teach models structured instruction-following.

Typical Uses:

- **Instruction Following**
  (e.g., summarization, explanation, transformation tasks)

- **Task-Oriented Assistants**
  (clear input → structured output)

- **Domain Adaptation**
  (e.g., legal, medical, coding — when combined with domain data)

Advantages

- **Task Versatility**
  Learns how to perform tasks, not just predict text

- **Structured Learning**
  Clear Instruction → Input → Output format improves consistency

- **Efficient Training**
  Works well with small datasets and LoRA

- **Alignment Improvement**
  Makes base models more helpful and controllable

Limitations

- **Synthetic Data Bias**
  Not fully representative of real user behavior

- **No Multi-Turn Learning**
  Cannot handle conversations alone

- **Data Quality Sensitivity**
  Poor samples → poor model performance

- **No Guarantee of Accuracy**
  Especially critical in domains like legal/medical

- **Limited Real-World Generalization**

Best Models

- Llama 3 / 3.1 (Base)

- Mistral 7B

- Phi-3 / Phi-4
- Qwen 2.5

2)**ShareGPT Format (Conversation Messages):**

Format

```
[
  {
    "conversations": [
      {
        "from": "human",
        "value": "..."
      },
      {
        "from": "assistant",
        "value": "..."
      }
    ]
  }
]
```

When to Use

- Chat SFT (Supervised Fine-Tuning for chat models)
- Chatbot development
- Dialogue systems
- Context retention training

Advantages

- Multi-turn learning
- Realistic interaction
- Context awareness
- Better conversation flow

Limitations

- Noisy data
- Complex preprocessing
- Longer sequences (higher compute cost)
- Privacy concerns

Best Models

- Llama 3 Instruct
- Mistral Instruct
- Vicuna
- Qwen Chat

3) **Completion Format (Raw Text)**:

Format:

```
{
 "text": "Long document or raw text"
}
```

When to Use:

Used for **pretraining or continued pretraining**.

Typical uses:

- Base model training
- Domain adaptation
- Language modeling

Advantages

- Massive datasets possible
- No annotation needed

Limitations

- No instruction-following behavior
- No conversational structure

Models that perform best with completion format datasets

These models are designed for **causal language modeling**:

- LLaMA 3 (excellent for continued pretraining)

- Mistral 7B (very efficient and fast training)
- GPT-3 (classic completion-style training)
- Falcon 40B

Why these models work well

They are trained using **next-token prediction**, which perfectly matches the **completion format**.

Advantages:

- large corpus support
- efficient training
- strong base knowledge

4) **DPO Format (Direct Preference Optimization):**

Structure:

```
{
 "prompt": "...",
 "chosen": "...",
 "rejected": "..."
}
```

Example:

```
{
 "prompt": "Explain gravity",
 "chosen": "Gravity is a force that attracts objects...",
 "rejected": "Gravity is when things fall down."
}
```

When to Use

Used in **alignment training**.

Typical uses:

- Preference learning
- Improving answer quality
- Replacing RLHF reward models

Advantages

- Improves response quality
- Teaches the model preferred answers

Limitations

- Requires human preference labeling
- More complex training

Models that perform best with DPO

- LLaMA 2 Chat
- Mistral 7B Instruct
- Claude
- GPT-4

Why these models work well

DPO requires a **chat/instruction model** because it learns **preference ranking between responses**.

The training process compares: (good answer vs bad answer)

This improves:

- safety

- answer quality
- reasoning

Which Dataset Format Processes Faster?

Processing speed depends on **structure complexity** and **tokens per sample**.

Rank	Format	Speed	Reason
1	Completion	Fastest	raw text, minimal parsing
2	Alpaca	Fast	simple instruction structure
3	ShareGPT	Medium	multi-message conversation
4	DPO	Slowest	requires pairwise comparison

1. Completion (Fastest)

Structure is simple:

text → tokens → prediction

Advantages:

- minimal preprocessing
- sequential training
- large batch efficiency

---

2. Alpaca

Structure:

instruction + input → output

Still simple, but requires **prompt formatting**.

---

3. ShareGPT

Structure:

message1
message2
message3
...

Slower because:

- multi-turn context
- longer sequences
- more tokens

---

4. DPO (Slowest)

Structure:

prompt
chosen
rejected

Training must compute:

log probability(chosen)
log probability(rejected)

So every sample requires **two forward passes**.

Formatting Functions

In an LLM training architecture, formatting functions are the scripts or functions that convert raw datasets into the required training format (Alpaca, ShareGPT, Completion, or DPO).
They ensure the model receives consistent prompts, tokens, and structures before training.

Hugging Face Dataset Object

A **Hugging Face Dataset object** is the core data structure used by the Hugging Face Datasets library to **store, process, and stream datasets efficiently for machine learning and LLM training**.

It is commonly used with frameworks from Hugging Face and integrates with libraries like Transformers.

Model Loading

Quantization:

Quantization is a technique used in machine learning and LLM architecture to **reduce the numerical precision of model weights and activations** so the model **uses less memory and runs faster**.

Instead of storing numbers in **32-bit floating point**, quantization converts them to **lower precision formats** like 16-bit, 8-bit, or 4-bit.

This technique is widely used in frameworks from Hugging Face and models like LLaMA 3.

1. Why Quantization Is Important

Large language models are very large.

Example:

Model	Memory (FP32)
7B parameters	~28 GB
13B parameters	~52 GB
70B parameters	~280 GB

Quantization reduces this dramatically.

Example:

Precision	Memory Reduction
FP16	2× smaller
INT8	4× smaller
INT4	8× smaller

Some Examples of Quantization are:

1) FP16 / BF16 (Half Precision)

Used in most training systems.

Advantages:

- Faster GPU computation
- Minimal accuracy loss

Example:

float32 → float16

Used with models like GPT-4.

---

2) 8-bit Quantization

Weights stored as **INT8 values**.

Benefits:

- 75% memory reduction
- Good performance

Common library:

- bitsandbytes

Example:

load_in_8bit=True

---

3) 4-bit Quantization

Very popular for **fine-tuning large models on consumer GPUs**.

Technique:

QLoRA

Associated with models like Mistral 7B.

Memory example:

Model	Normal	4-bit
7B model	~28 GB	~3.5 GB

<u>If Quantization is enabled</u>:

<u>QLoRA (Quantized Low-Rank Adaptation)</u>

Concept

QLoRA (Quantized Low-Rank Adaptation) is an advanced **Parameter Efficient Fine-Tuning (PEFT)** technique that combines **4-bit quantization** with **LoRA adapters** to enable fine-tuning of large language models with extremely low GPU memory usage.

Instead of loading the base model in full precision, the model weights are **quantized to 4-bit**, while **LoRA adapters are trained in higher precision** to adjust the model's behavior.

The base model remains **frozen and quantized**, and only the small **LoRA adapter parameters** are updated during training.

This method enables fine-tuning large models such as LLaMA 2 and Mistral 7B on consumer-level GPUs.

Workflow

Base Model → 4-bit Quantization → Freeze Weights → Inject LoRA Adapters → Train Adapters → Merge or Use Separately

Base model is loaded

The pretrained model is loaded in **4-bit quantized format** to reduce memory usage.

Original weights are frozen

The quantized model weights remain **frozen and unchanged** during training.

LoRA adapters are added

Small **low-rank adapter matrices** are injected into transformer layers.

Only adapters are trained

Training updates only the **LoRA adapter parameters**, not the full model.

Final model can:

- use adapters separately (recommended for modularity)
- merge adapters into the base model for standalone inference

---

When to Use

Low to medium GPU availability (**8GB–24GB GPUs**)

Fine-tuning **large models (7B–70B)** with limited hardware

Need **extremely memory-efficient training**

Training on consumer GPUs such as RTX 3090 / RTX 4090

Running multiple experiments without large compute clusters

Advantages

Extremely **low memory usage** (up to ~90% reduction)

Enables fine-tuning large models on **consumer GPUs**

Cost-efficient training

Maintains strong performance close to full fine-tuning

Compatible with modern libraries like bitsandbytes and PEFT

Supports modular adapter-based architecture

Limitations

Slightly slower training due to **quantized computations**

Dependent on **base model quality**

May introduce **small accuracy loss** compared to full precision training

More complex setup than standard LoRA

Hardware optimization mainly designed for **NVIDIA GPUs**

<u>If Quantization is not enabled</u>:

<u>LoRA (Low-Rank Adaptation)</u>

Concept

LoRA (Low-Rank Adaptation) is a **Parameter Efficient Fine-Tuning (PEFT)** technique where, instead of updating the entire model, only a **small set of additional trainable parameters (LoRA adapters)** are learned.

The base model remains **frozen**, and LoRA learns a **low-rank update** to adjust model behavior efficiently.

Workflow

Base Model → Freeze Weights → Inject LoRA Layers → Train Adapters → Merge or Use Separately

- Base model is loaded
- Original weights are frozen

- LoRA layers (low-rank matrices) are added
- Only these small layers are trained
- Final model can:
    - use adapters separately OR
    - merge into base model

When to Use

- Medium GPU availability (16GB+)
- Need faster and cost-efficient fine-tuning
- Working with 7B–13B models
- When base model is already strong and only needs behavior adjustment

Advantages

- Reduced training cost
- Faster training
- Memory efficient
- Preserves base model knowledge
- Modular and flexible (adapters can be reused or merged)

Limitations

- Limited learning capacity (cannot deeply learn completely new knowledge)
- Dependent on base model quality
- Requires moderate GPU
- Sensitive to hyperparameters (rank, alpha, etc.)

<u>Training method</u>

SFT (Supervised Fine-Tuning)

Concept

SFT (Supervised Fine-Tuning) is the process of training a pretrained model on **labeled input–output pairs**, where the model learns to generate the correct response for a given instruction or prompt.

It is the **first stage of alignment**, teaching the model how to follow instructions.

Workflow:

Pretrained Model → Load Instruction Dataset → Tokenization → Train Model with Prompt-Response Pairs → Fine-Tuned Model

- Dataset is prepared in structured format
- Model learns from correct examples
- Loss is computed between predicted output and ground truth
- Model weights (or adapters) are updated

When to Use

- Initial stage of fine-tuning

- Teaching instruction-following behavior
- Domain adaptation (legal, medical, etc.)
- Before applying alignment methods like DPO

Advantages

- Simple and stable training process
- Direct control over model behavior
- Works well with Alpaca and ShareGPT datasets
- Strong baseline for further alignment

Limitations

- Depends heavily on dataset quality
- Cannot distinguish between “good” and “better” answers
- May produce generic or average responses
- Does not optimize for preference or ranking

Key Insight

SFT teaches the model **what to say**,
but not **which answer is better**.

DPO (Direct Preference Optimization)

Concept

DPO (Direct Preference Optimization) is an alignment technique where the model is trained using **pairs of responses (chosen vs rejected)** to learn which outputs are preferred.

It replaces traditional RLHF by directly optimizing preference without training a reward model.

Workflow: Pretrained/SFT Model → Preference Dataset (Prompt + Chosen + Rejected) → Compare Responses → Optimize Preference Loss → Aligned Model

- Each sample contains:
  - prompt
  - chosen (better response)
  - rejected (worse response)
- Model learns to assign higher probability to better responses

---

When to Use

- After SFT
- Improving response quality
- Aligning model with human preferences
- Reducing hallucinations and unsafe outputs

Advantages

- Improves response quality and ranking
- Simpler than RLHF (no reward model needed)
- More stable training compared to reinforcement learning
- Better alignment with human expectations

Limitations

- Requires high-quality preference data
- Data creation is expensive (human labeling or GPT-4 generation)
- More complex than SFT
- Dependent on quality of chosen/rejected pairs

Key Insight

DPO teaches the model **which answer is better**,
not just how to generate an answer.

ORPO (Odds Ratio Preference Optimization)

Concept

ORPO (Odds Ratio Preference Optimization) is a newer alignment method that combines **SFT and preference optimization into a single training stage**.

It improves model alignment by optimizing both:

- likelihood of correct responses
- preference ranking

Workflow: Pretrained Model → Instruction + Preference Dataset → Joint Optimization (SFT + Preference) → Aligned Model

- No separate SFT + DPO stages
- Uses preference signals during training
- Simplifies pipeline

When to Use

- When you want a **simpler training pipeline**
- Replacing SFT + DPO with a single step
- Resource-constrained environments
- Faster experimentation

Advantages

- Combines SFT and preference learning
- Reduces training complexity
- Faster pipeline (single-stage training)
- Good balance between performance and efficiency

Limitations

- Less mature compared to DPO
- Fewer implementations and community support
- Requires careful tuning
- May not outperform DPO in all cases

Key Insight

ORPO tries to **merge learning and alignment into one step**,
reducing pipeline complexity while maintaining performance.

Final Understanding (Very Important)

SFT → teaches how to respond
DPO → teaches which response is better
ORPO → combines both in one training step